Name: Piyush Gupta

Roll No: 2401033

Class: TE, Batch: T12

Subject: ASD&D

Experiment : 5

Aim: To automate application deployment using Github/Jenkins

Lab Outcome: Mapped LO3

LO3: Build and manage automated CI/CD pipelines for application development, testing, packaging and deployment

Theory:

Continuous Integration and Continuous Deployment

CI/CD is a software development approach used to automate the movement of source code from development to a running application. It reduces repetitive manual work and allows changes to be validated and delivered more consistently.

● Continuous Integration (CI):

Continuous Integration is the practice of frequently integrating developers' changes into a shared source-code repository. Whenever new code is introduced, automated processes can compile, validate, and test the changes. This helps identify errors at an early stage and maintains a stable codebase.

● Continuous Deployment (CD):

Continuous Deployment extends the CI process by automatically delivering successfully validated changes to a target environment. Once the pipeline completes the required checks, the latest application files can be transferred to the deployment location without manually copying the files.

● Jenkins:

Jenkins is an open-source automation server that can coordinate activities such as building, testing, and deploying software. It can obtain source code from repositories such as GitHub and execute a predefined sequence of pipeline stages.

● Jenkins Declarative Pipeline:

A Declarative Pipeline describes the CI/CD workflow in a structured form using a Jenkinsfile. The file can contain stages such as Build, Test and Deploy. Keeping the pipeline definition with the source code makes the automation process easier to reproduce and maintain.

● GitHub:

GitHub provides remote version control for the application source code. In this experiment, GitHub acts as the source repository from which Jenkins retrieves the latest version of the application.

● CI/CD Workflow:

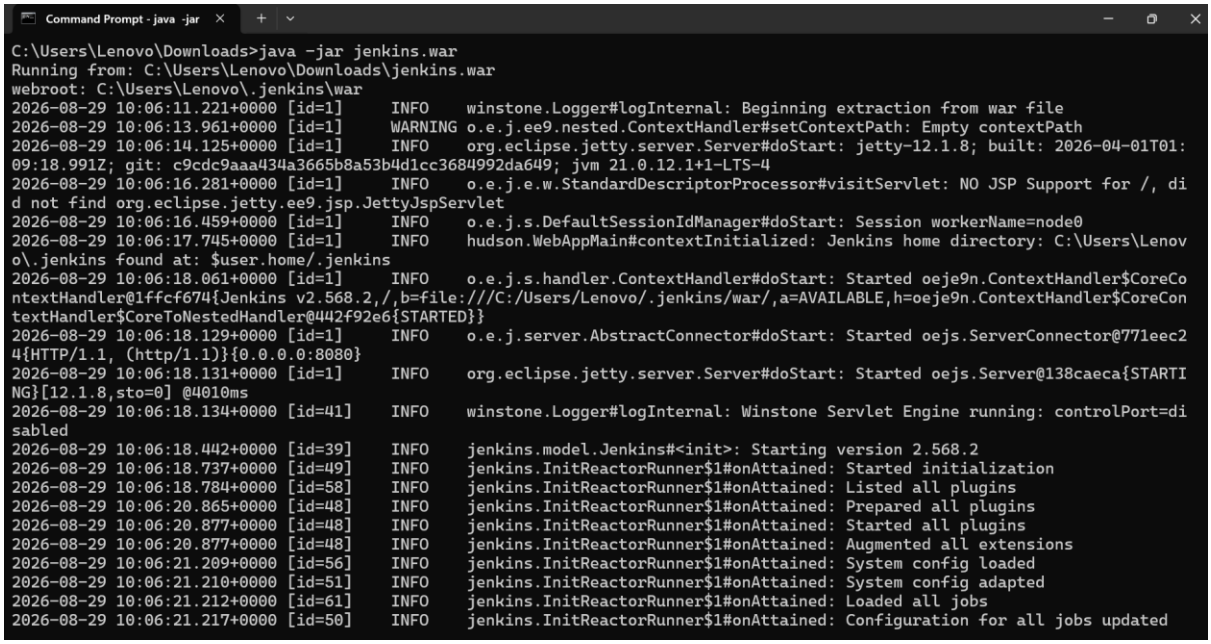
The workflow implemented in this experiment can be represented as:

Developer changes code → GitHub repository → Jenkins retrieves code → Build → Test → Deploy → Updated application

EXECUTION:

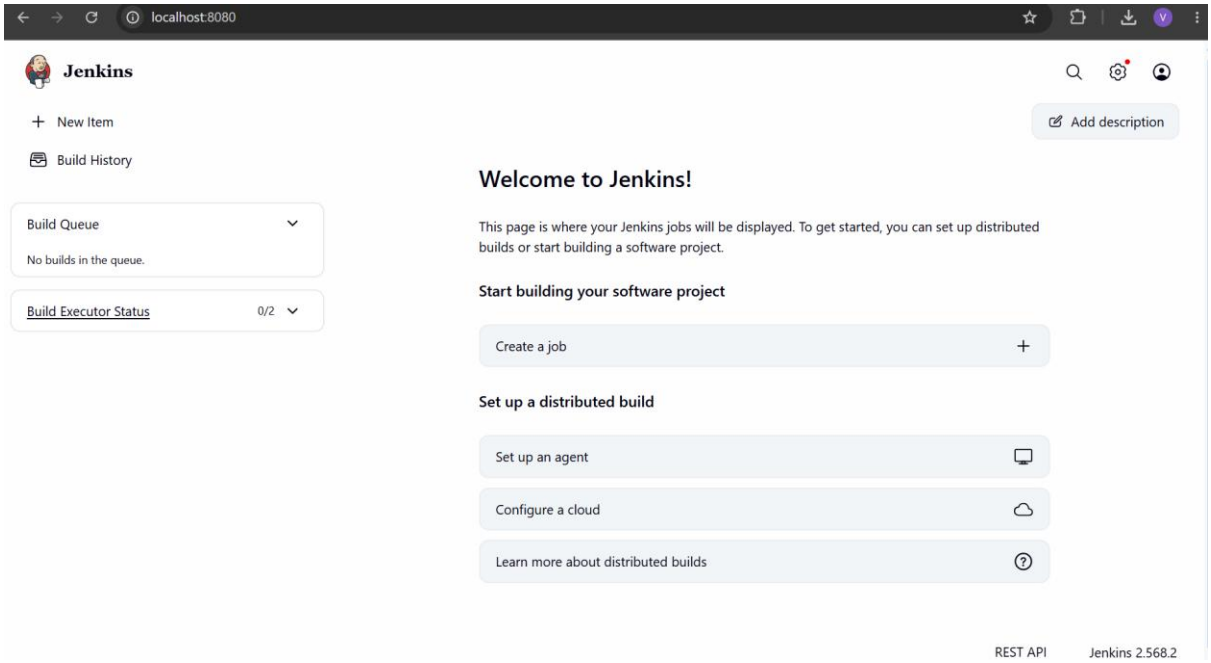
Step 1: Jenkins Initialization and Setup

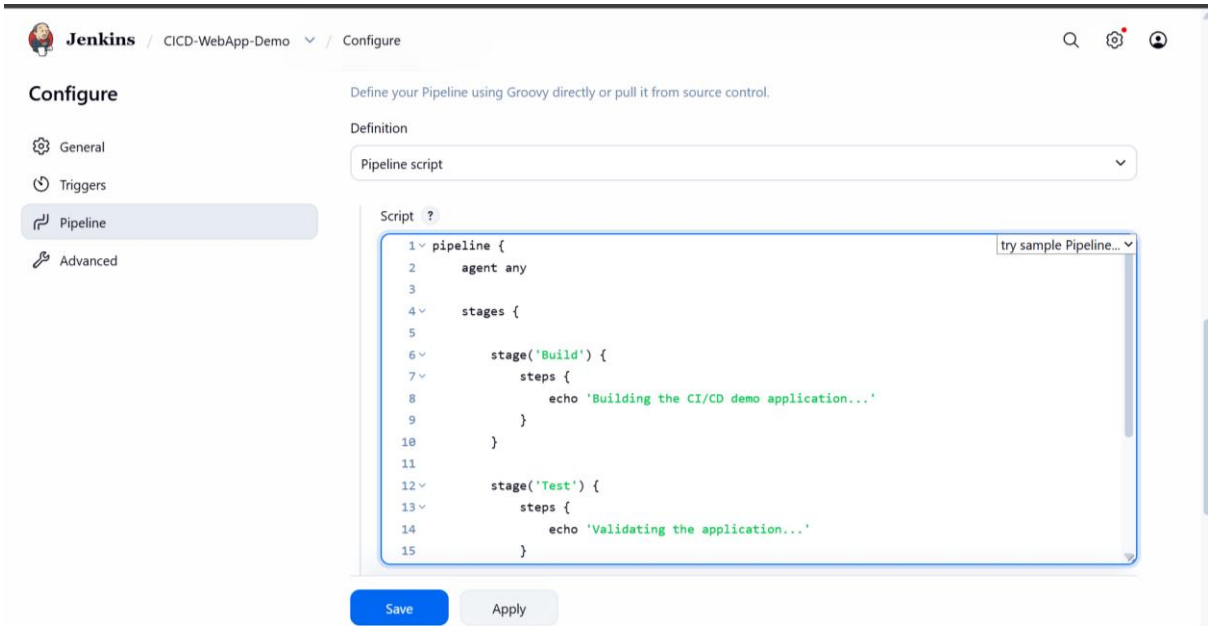
Jenkins was configured on the Windows system using a compatible Java runtime. The installed JDK 21 environment was verified before starting the Jenkins server.



Step 2: Jenkins Pipeline Creation and Initial Validation

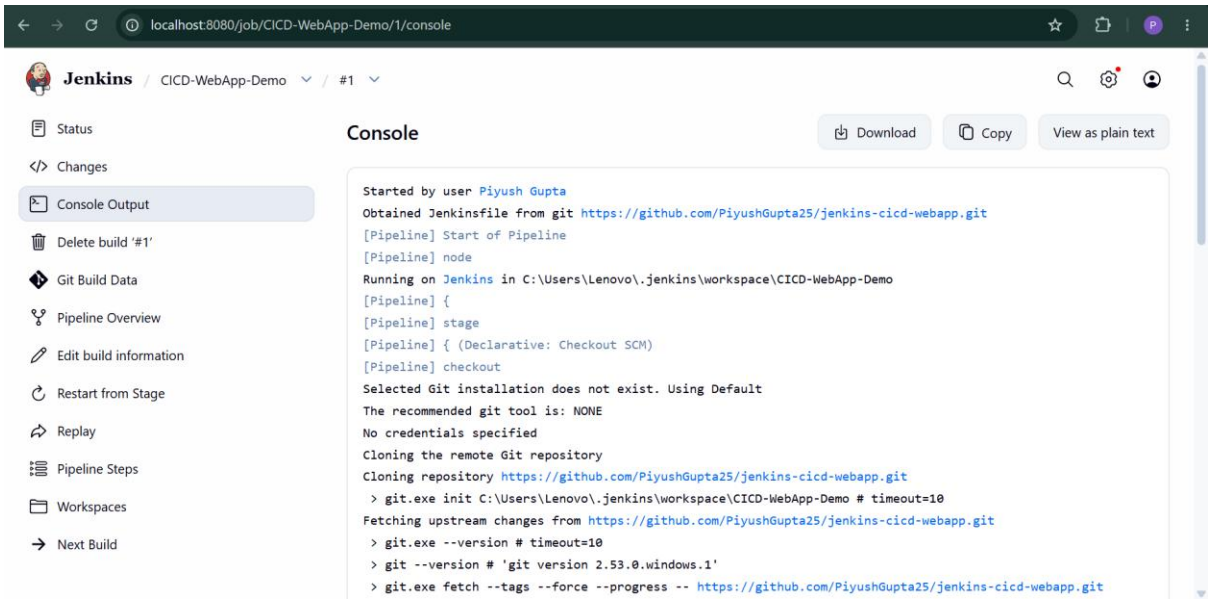
Jenkins was accessed through the local web interface and a new Pipeline project named CICD-WebApp-Demo was created. A Declarative Pipeline containing Build, Test and Deploy stages was configured to verify the Jenkins execution environment.





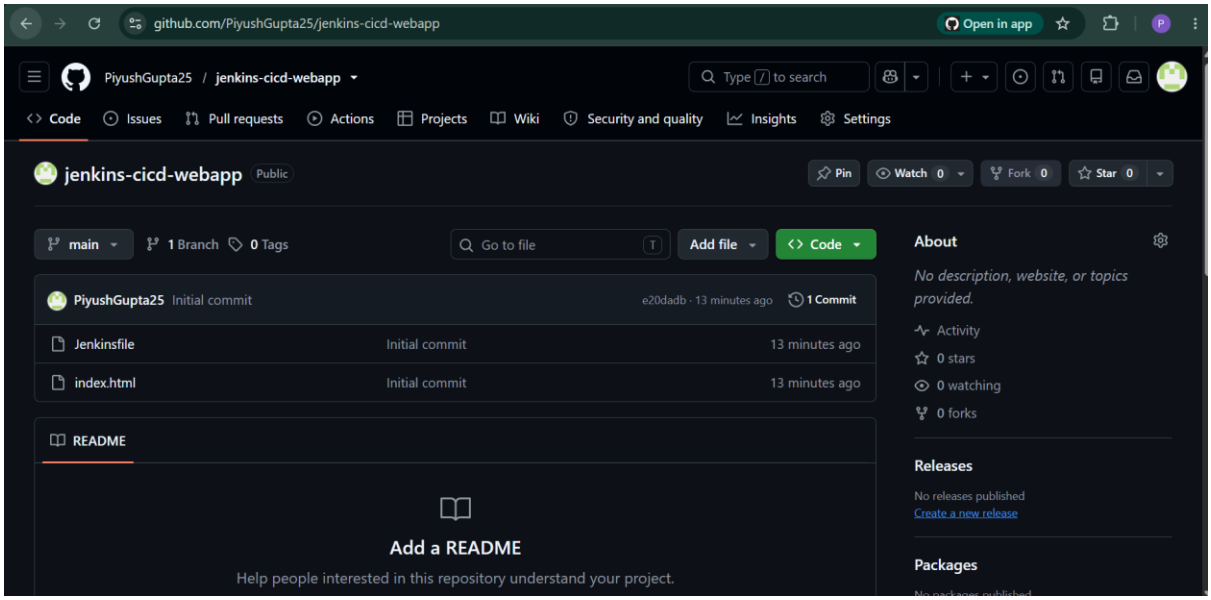
Step 3: Initial Pipeline Execution

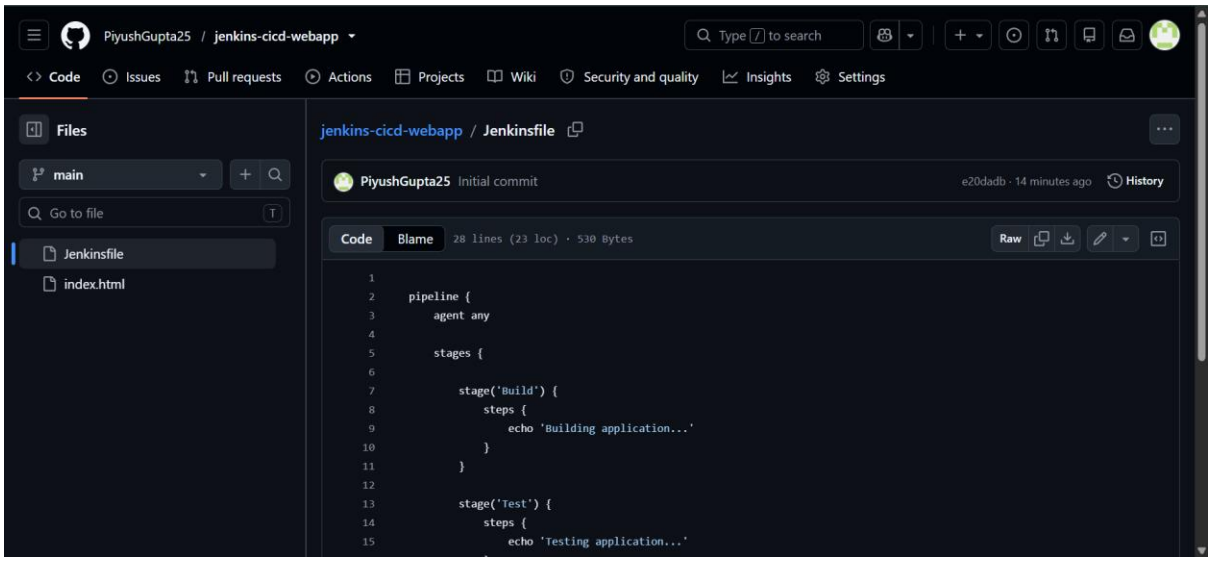
The configured pipeline was executed using Jenkins. The Build, Test and Deploy stages were processed sequentially, and the console output was checked to confirm successful completion.



Step 4: GitHub Repository and Jenkinsfile Configuration

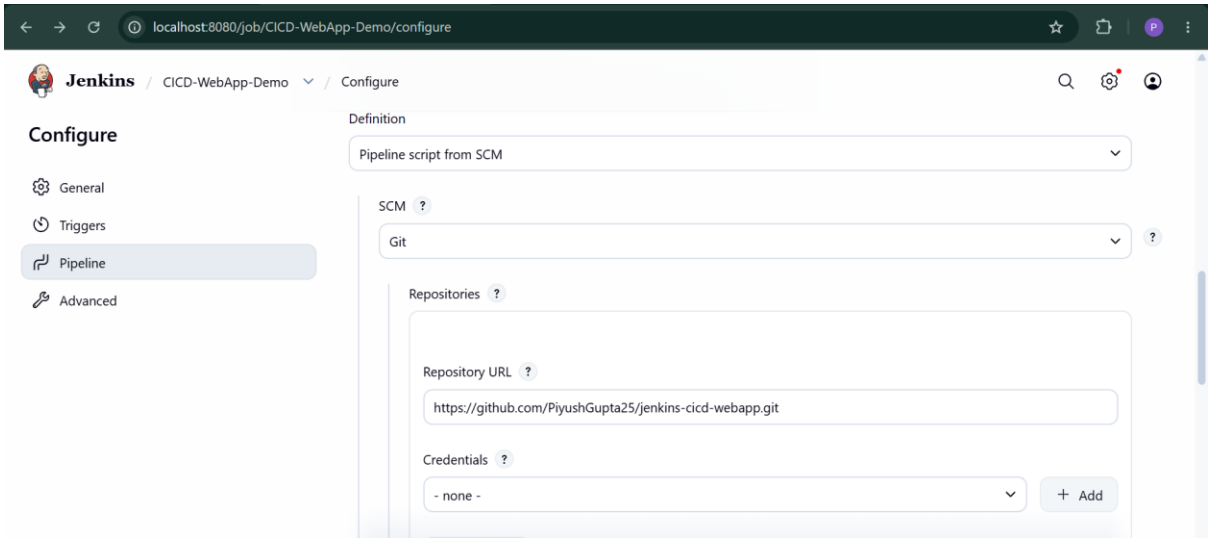
A GitHub repository named jenkins-cicd-webapp was created to store the application source code and Jenkins pipeline definition. The repository was populated with the index.html application and a Jenkinsfile containing the automated Build, Test and Deploy stages.

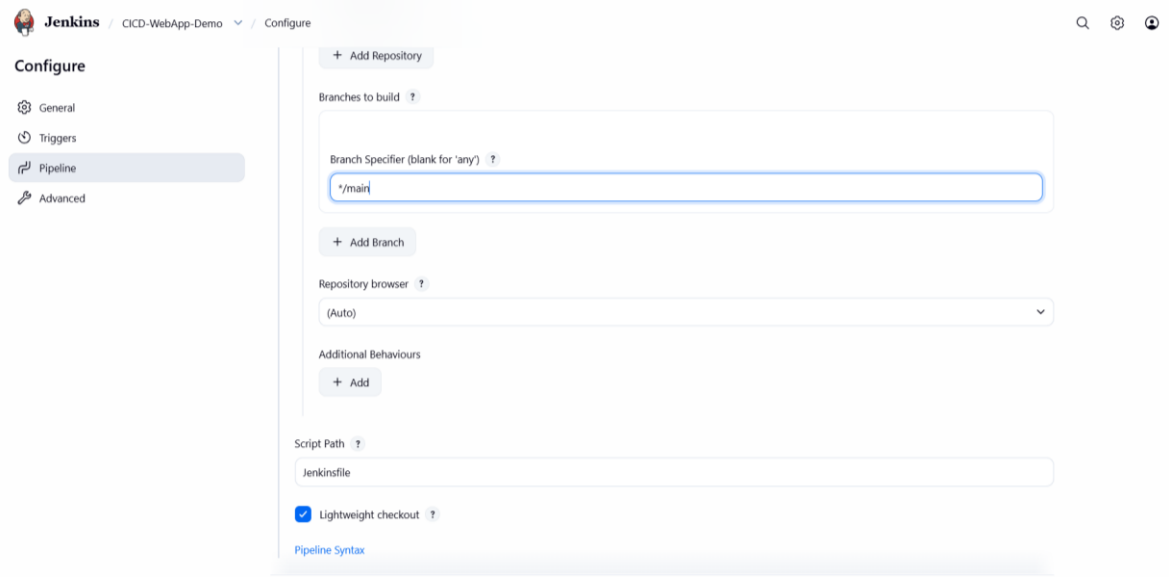




Step 5: Jenkins-GitHub Integration

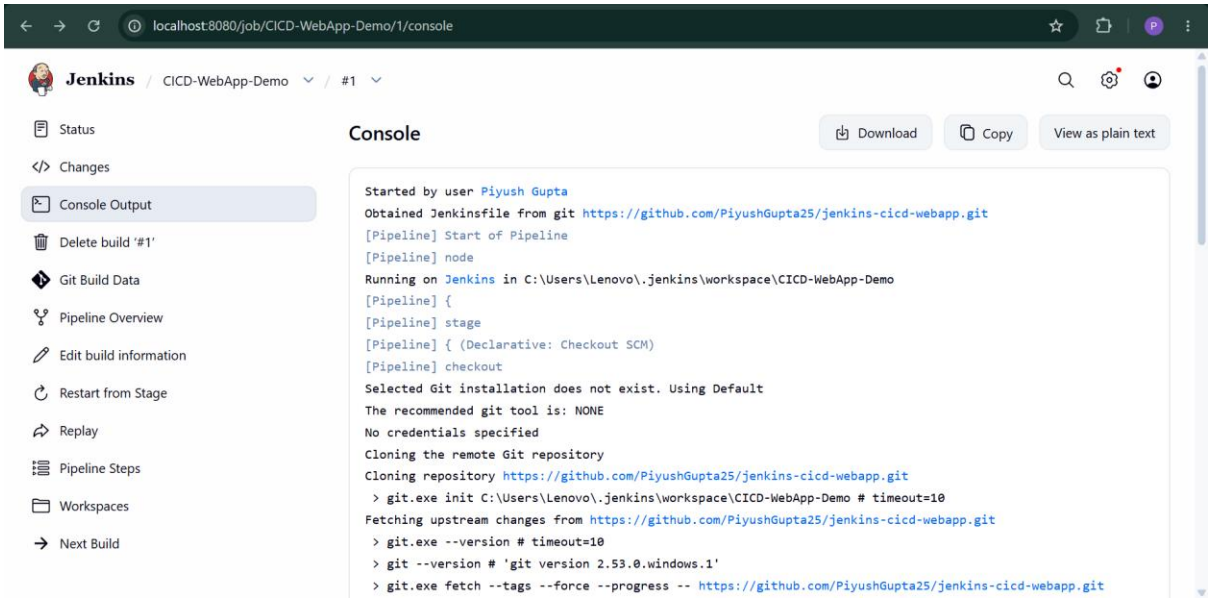
The Jenkins project was modified to obtain its pipeline definition directly from the GitHub repository. Git was selected as the source control mechanism, the repository URL and main branch were configured, and Jenkinsfile was specified as the pipeline script.

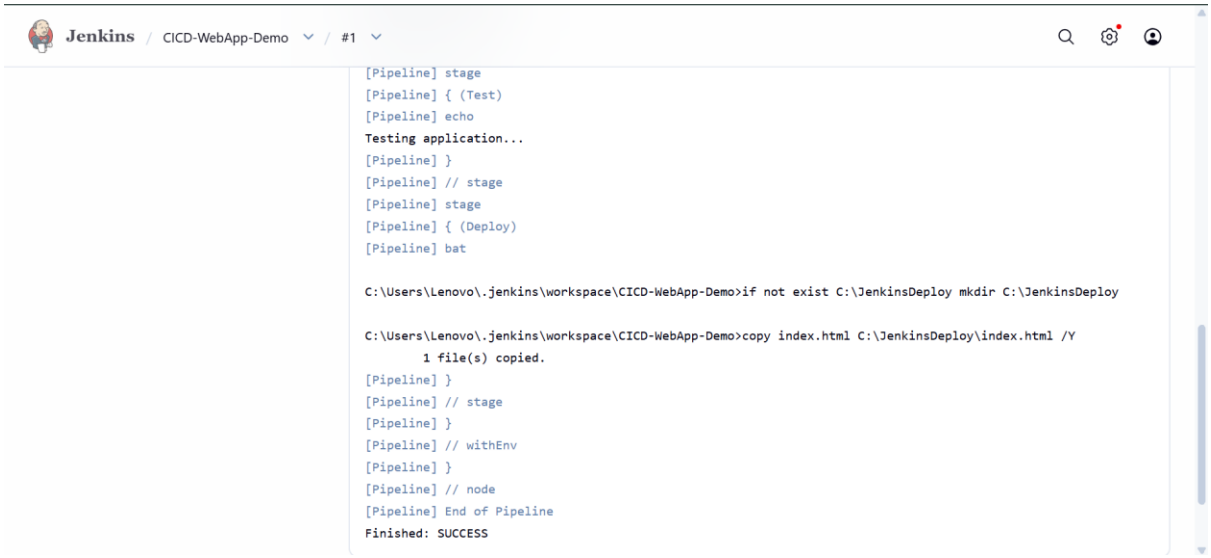




Step 6: GitHub Checkout and Automated Deployment

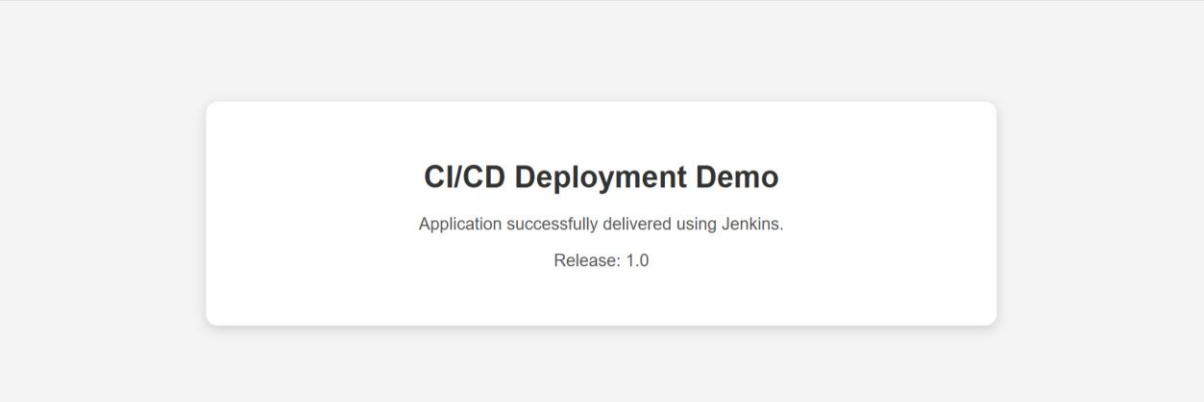
Jenkins was executed after configuring the GitHub repository. The system retrieved the latest source code, loaded the Jenkinsfile and executed the defined pipeline stages. During the Deploy stage, the application file was copied to the configured local deployment directory.





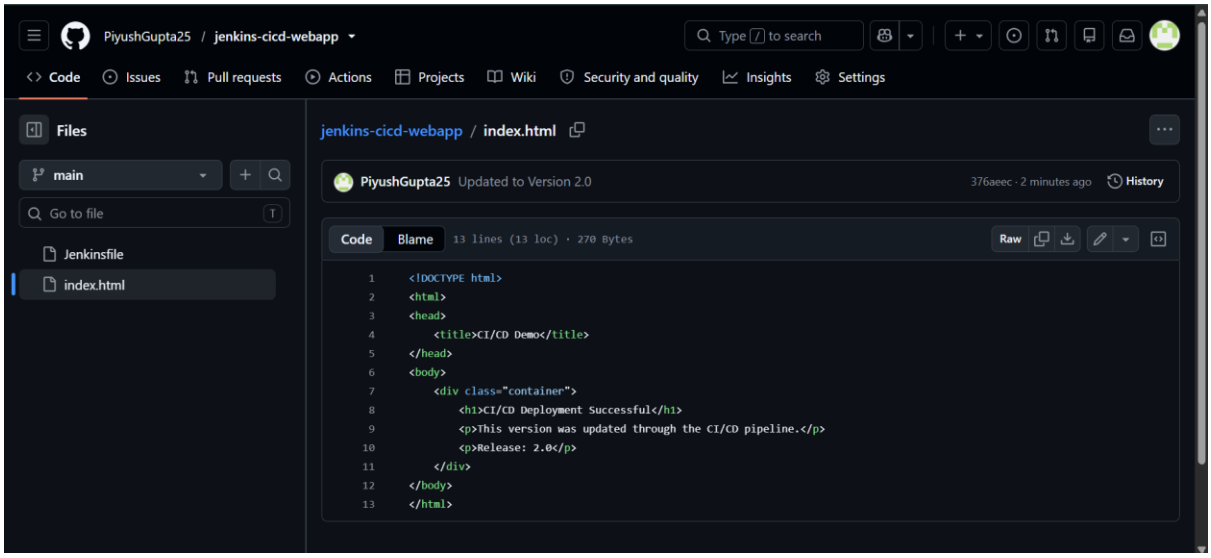
Step 7: Initial Application Deployment

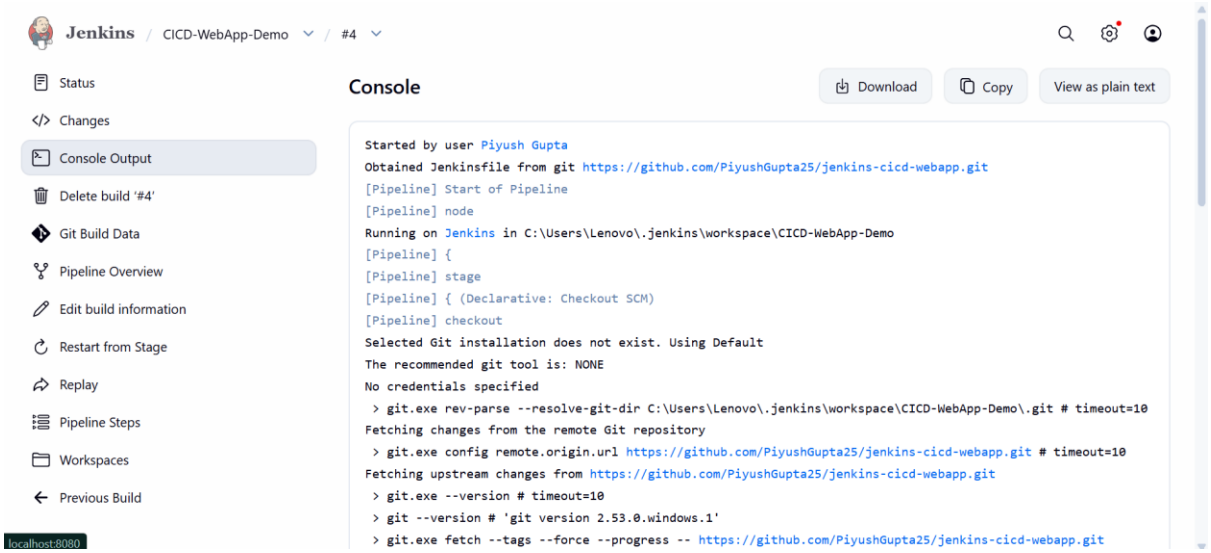
After successful pipeline execution, the generated application file was verified in the local deployment directory. The index.html file was opened in a web browser to confirm that the first version had been deployed correctly.



Step 8: CI/CD Update and Redeployment

The application source code was modified in the GitHub repository to create a new release. The updated version was committed and the Jenkins pipeline was executed again so that the latest source code could be retrieved and deployed.







Step 9: Final Deployment Verification

The deployed application was refreshed after the successful Jenkins execution. The updated Version 2.0 content was displayed, confirming that the modification made in GitHub had successfully passed through the CI/CD pipeline and reached the deployment environment.



Conclusion:

The experiment demonstrated the practical implementation of a CI/CD workflow by integrating GitHub with Jenkins. The use of a Declarative Jenkinsfile allowed the Build, Test and Deploy operations to be defined as a repeatable pipeline rather than being performed manually. The successful deployment of both the initial and modified application versions showed how source-code changes can be transferred from the repository to the target environment through an automated workflow. This experiment therefore provided practical understanding of Pipeline as Code, source-code integration, automated deployment and consistent software delivery.